

1 Development

1.1 The heuristic of A*

1.1.1 What A* needs from a heuristic

A* expands first the node with the smallest value

$$f(n) = g(n) + h(n), \quad (1)$$

where $g(n)$ is the number of moves already made to reach n and $h(n)$ is an estimation of the moves that are still missing to reach the solved cube [1, 2]. A good introduction with visual examples can be found in [3]. The quality of h decides how many nodes A* has to visit, and for the Rubik's Cube this is critical: the cube has 43,252,003,274,489,856,000 reachable states [4, 5], every state has 18 possible moves (quarter and half turns of the six faces), and even the hardest cubes need 20 moves [6].

Two properties are important [1, pp. 94–95]:

- **Admissible:** $h(n) \leq h^*(n)$, where $h^*(n)$ is the real number of moves needed from n . The heuristic never overestimates.
- **Consistent:** $h(n) \leq c(n, a, n') + h(n')$ for every move a that goes from n to n' . Since every move costs $c = 1$, this means that h changes at most by 1 after one move.

If h is consistent (and therefore admissible), A* returns a shortest solution and a state never needs to be expanded twice [1, p. 95]. We need both properties in our program.

1.1.2 First version of the heuristic and its problems

Our first Python implementation used five small tables (explained below), each one storing information about only 4 cubies: two tables for 4 corners ($8 \cdot 7 \cdot 6 \cdot 5 \cdot 3^4 = 136,080$ states each) and three tables for 4 edges ($12 \cdot 11 \cdot 10 \cdot 9 \cdot 2^4 = 190,080$ states each), 842,400 entries in total. The heuristic was the maximum of the five values. It was correct, but it had two problems:

1. **It is weak.** Each table ignores 8 of the 20 movable cubies, so its values are small. We measured the average value of h over 20,000 random cubes and obtained only 6.45 moves, while a typical random cube needs 17 or 18 moves [5]. A weak heuristic makes A* expand a huge number of nodes.
2. **It was built for 12 moves.** In the Python version only quarter turns existed. In this project we also use half turns ($F2, U2, \dots$), so there are 18 moves. A table built with quarter turns counts a half turn as 2 moves, so it can overestimate when the search counts it as 1 move, and the heuristic would stop being admissible. Every table must be built with exactly the same moves as the search.

1.1.3 Pattern databases

The idea of a *pattern database* (PDB) was introduced by Culberson and Schaeffer [7] and it is also explained in our course book [1, p. 106]. We choose a part of the problem (for example, only the 8 corners of the cube and we ignore the edges). The number of moves needed to solve only that part can never be larger than the number of moves needed to solve the whole cube, so it is an admissible heuristic. Since that smaller problem has few states, we can solve *all* its states in advance and save the answers in a table. During A*, computing $h(n)$ is only one table look-up.

The table is built with a breadth-first search that starts at the solved cube and applies the 18 moves again and again. The first time that the search reaches an arrangement of the pieces, the current depth is the distance of that arrangement to the solved state (the moves can be undone, so the distance from the solved cube is the same as the distance to it).

1.1.4 Heuristic selected: 8 corners and two groups of 6 edges

For the final version we use the heuristic of Korf [4], who solved random cubes optimally with it. It uses three pattern databases:

1. all 8 corners: $8! \cdot 3^7 = 88,179,840$ entries (the last corner twist is determined by the other seven, hence 3^7);
2. 6 edges (the first six edge cubies): $\frac{12!}{6!} \cdot 2^6 = 42,577,920$ entries;
3. the other 6 edges: also 42,577,920 entries.

The heuristic is the maximum of the three values:

$$h(n) = \max(h_{\text{corners}}(n), h_{\text{edges A}}(n), h_{\text{edges B}}(n)). \quad (2)$$

Table 1 shows the tables. Korf stored 4 bits per entry (82 MB in total); to keep our code simple we store one byte per entry (173 MB), which is small for our cluster. The tables are built once (about 12 seconds in our tests with 2 threads) and they are saved in a file for the next executions. As a check of our implementation, the average of the corner table is 8.764, exactly the value reported by Korf [4].

Table	Entries	Memory	Average h	Max. h
8 corners	88,179,840	88.2 MB	8.764	11
6 edges, set A	42,577,920	42.6 MB	7.623	10
6 edges, set B	42,577,920	42.6 MB	7.614	10

Table 1: Pattern databases used by A* (averages over all the entries of each table, measured with our program; Korf reports 8.764 for the corners and 7.668 for his 6-edge table [4]).

Why the maximum and not the sum. It would be tempting to add the three values to get a bigger heuristic, but this is not admissible for the cube: one turn of a face moves 4 corners *and* 4 edges at the same time, so the same move would be counted in two tables. Adding values is only valid when the tables count different moves, as in additive pattern databases for sliding-tile puzzles [8]; the book also remarks that this does not work for the Rubik’s Cube [1, p. 107]. Korf states that the only admissible way to combine corner and edge tables is to take the maximum [4], and Holte et al. studied experimentally the use of several tables with the maximum [9]. We can also prove that the maximum keeps the two properties. Let $h_1, \dots, h_k$ be admissible and consistent heuristics and $h = \max_i h_i$:

- *Admissible*: for each i , $h_i(n) \leq h^*(n)$, so $\max_i h_i(n) \leq h^*(n)$.
- *Consistent*: for each i , $h_i(n) \leq 1 + h_i(n') \leq 1 + h(n')$. Taking the maximum over i on the left side gives $h(n) \leq 1 + h(n')$.

Each table is itself consistent because it is the exact distance in a smaller version of the same puzzle, where one real move changes the pattern by at most one move. Our program also checks both properties in a self-test with thousands of random cubes.

How much better is it? Figure 1 compares the average value of h of the first version and of the final version over 20,000 random cubes for different scramble lengths. For a completely mixed cube the average goes from 6.45 to 8.89 moves (Korf reports 8.878 [4]). It seems a small difference, but A* has to visit all the nodes with $f(n) = g(n) + h(n)$ smaller than the solution length, and the number of nodes grows

exponentially with the depth, so every extra move of the heuristic removes a large part of the search. Korf also observed that the running time goes down approximately in proportion to the size of the tables [4]; our final tables have 173,335,680 entries, about 206 times more than the 842,400 entries of the first version.

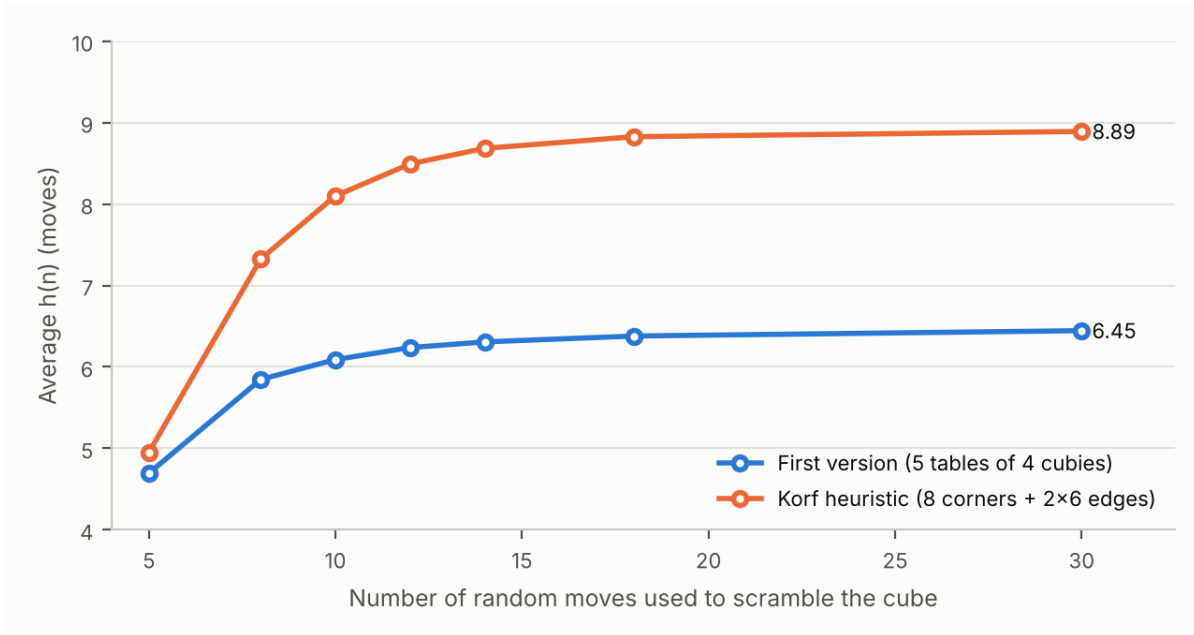


Figure 1: Average value of h for random cubes scrambled with different numbers of moves (20,000 cubes per point). Higher is better, as long as it is admissible.

1.1.5 Implementation details

- **Index of a table.** A pattern is converted to a number: the positions of the tracked cubies are encoded as a permutation number (Lehmer code), and the twists or flips of the cubies as a number in base 3 (corners) or base 2 (edges).
- **Open list.** Since every move costs 1, f is always a small integer. Instead of a heap, the open list of our C++ A* is an array of lists, one list for every value of f . Inside a list, the newest node is expanded first (it has a larger g , so it is closer to the goal).
- **Redundant moves.** A face is never turned twice in a row, because two consecutive turns of the same face are always equal to one single move.
- **Memory.** Korf used IDA* [10], a version that needs almost no memory, because A* stores every node it generates. We keep A* because it is the algorithm requested in this project, so we expect that for the deepest scrambles the memory of the cluster will be the limit; this will be analyzed in the Results.

References

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2010.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968, doi: 10.1109/TSSC.1968.300136.

- [3] A. Patel, “Introduction to the A* algorithm,” Red Blob Games, 2014. [Online]. Available: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>. Accessed: Oct. 2026.
- [4] R. E. Korf, “Finding optimal solutions to Rubik’s Cube using pattern databases,” in *Proc. 14th Nat. Conf. Artif. Intell. (AAAI-97)*, Providence, RI, USA, 1997, pp. 700–705.
- [5] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “God’s number is 20.” [Online]. Available: <https://www.cube20.org/>. Accessed: Oct. 2026.
- [6] T. Rokicki, H. Kociemba, M. Davidson, and J. Dethridge, “The diameter of the Rubik’s Cube group is twenty,” *SIAM J. Discrete Math.*, vol. 27, no. 2, pp. 1082–1105, 2013, doi: 10.1137/120867366.
- [7] J. C. Culberson and J. Schaeffer, “Pattern databases,” *Comput. Intell.*, vol. 14, no. 3, pp. 318–334, Aug. 1998, doi: 10.1111/0824-7935.00065.
- [8] A. Felner, R. E. Korf, and S. Hanan, “Additive pattern database heuristics,” *J. Artif. Intell. Res.*, vol. 22, pp. 279–318, 2004, doi: 10.1613/jair.1480.
- [9] R. C. Holte, A. Felner, J. Newton, R. Meshulam, and D. Furcy, “Maximizing over multiple pattern databases speeds up heuristic search,” *Artif. Intell.*, vol. 170, no. 16–17, pp. 1123–1136, 2006, doi: 10.1016/j.artint.2006.09.002.
- [10] R. E. Korf, “Depth-first iterative-deepening: An optimal admissible tree search,” *Artif. Intell.*, vol. 27, no. 1, pp. 97–109, 1985.